

Lista de Revisão para Prova 2 - Projeto Integrador, Testes e Qualidade

Objetivos de aprendizagem

- Revisar os conceitos da Parte 2 do curso, conectando POO, integração, banco de dados, exceções, padrões de projeto, testes e CI.
- Exercitar leitura crítica de arquitetura e código em `C++` e `Python` no contexto do mini-SCADA da estação de bombeamento.
- Preparar o aluno para responder questões introdutórias, médias e avançadas com argumentação técnica, não apenas memorização.

Tempo estimado: 4h

Vídeo de contexto

Antes de resolver esta lista, revise especialmente as aulas da Parte 2: cenário e modelagem, listas dinâmicas, validação de estado, bancos de dados, tratamento de erros, padrões de projeto, testes, CI e checklist profissional de entrega.

1. Como usar esta lista

- Esta lista prepara para a **Prova 2**, mas não é a prova.
 - O foco é demonstrar entendimento técnico por meio de explicação, análise de código, modelagem e decisões de projeto.
 - Em todas as respostas, priorize a estrutura `conceito -> impacto -> ação recomendada`.
 - Quando houver código, explique o problema antes de sugerir correção.
 - Quando houver arquitetura, conecte a decisão a POO, manutenção, testes e execução real.
 - Quando houver banco de dados, diferencie dado atual, histórico, transação e série temporal.
 - Quando houver testes, diga o que o teste prova e o que ele não prova.
-

2. Mapa de revisão da Parte 2

Tema	O que revisar	Evidência esperada na resposta
Modelagem do mini-SCADA	separação entre controlador C++ e supervisor Python	responsabilidade de cada camada
Contrato JSON	campos obrigatórios, tipos e validação	diferença entre JSON válido e contrato válido
Listas dinâmicas	<code>vector<unique_ptr<Sensor>></code> , listas Python, objetos heterogêneos	relação com polimorfismo
Encapsulamento e validação	estado protegido e leitura inválida	invariantes e exceções
Banco de dados	CSV, SQLite, tabelas, séries temporais	histórico consultável e rastreável
Exceções	<code>throw</code> , <code>try</code> , <code>catch</code> , <code>ValueError</code> , falha controlada	erro tratado sem derrubar o sistema
Padrões de projeto	Strategy, Command, Repository, Observer	problema que cada padrão reduz
Testes	unitário, contrato, integração, regressão, fumaça	matriz de testes e risco
CI e entrega	GitHub Actions, PR, README, AI_LOG	evidência reproduzível
Qualidade de código	coesão, acoplamento, responsabilidade, testabilidade	decisão técnica defensável

3. Questões introdutórias

Questão 1. Separação entre controlador e supervisor

No projeto da estação de bombeamento, o sistema foi dividido em:

- controlador/dispositivo em `C++` ;
- mini-SCADA em `Python/Streamlit` .

Explique:

1. qual é a responsabilidade principal de cada parte;
2. por que essa separação ajuda a entender o sistema;
3. qual risco aparece se o supervisor passar a alterar diretamente o estado interno do controlador;
4. como o contrato JSON reduz o acoplamento entre as linguagens.

Questão 2. Contrato JSON além da sintaxe

Considere o JSON abaixo.

```
{"tag": "LT-101", "valor": "alto"}
```

Responda:

1. esse texto pode ser JSON válido? Justifique;
2. por que ele não é suficiente como contrato para o mini-SCADA;
3. quais campos mínimos deveriam existir em uma leitura;
4. que tipo de teste deveria rejeitar esse exemplo.

Questão 3. Listas dinâmicas e polimorfismo

Explique por que uma estação com sensores de nível, pressão, vazão e temperatura se beneficia de uma lista dinâmica de sensores.

Na sua resposta, discuta:

1. por que `vector<unique_ptr<Sensor>>` faz sentido em `C++` ;
2. como a mesma ideia aparece em uma lista de objetos em `Python` ;
3. qual é a relação entre essa estrutura e polimorfismo;
4. qual erro conceitual ocorre quando a lista vira apenas um conjunto de strings e `if` .

Questão 4. Dados atuais, históricos e séries temporais

No mini-SCADA, a tela mostra o valor atual dos sensores, mas o projeto também exige histórico.

Explique:

1. a diferença entre valor atual e histórico;
2. por que leituras de sensores formam uma série temporal;
3. por que `timestamp` é parte essencial do contrato;
4. por que SQLite é suficiente para o projeto, mesmo não sendo um banco temporal especializado.

Questão 5. Exceção como falha controlada

Explique a diferença entre:

- erro ignorado;
- erro que derruba o programa;
- erro tratado de forma controlada.

Use como exemplo uma leitura de sensor fora da faixa esperada. Na resposta, conecte `throw / catch` em `C++` com `raise / except` em `Python`.

Questão 6. Teste unitário, contrato e integração

Compare os três testes abaixo no contexto do projeto.

Teste	Exemplo
Unitário	regra de nível baixo gera <code>LIGAR_BOMBA</code>
Contrato	JSON possui <code>tag</code> , <code>valor</code> , <code>unidade</code> , <code>timestamp</code> , <code>status</code>
Integração	Repository salva leitura no SQLite e consulta depois

Explique o que cada teste prova e qual limitação cada um possui.

4. Questões médias: análise crítica de código e projeto

Questão 7. Revisão de código em C++: lista sem polimorfismo

Analise o código.

```
std::vector<std::string> sensores = {"nivel", "pressao", "vazao"};

for (const auto& tipo : sensores) {
    if (tipo == "nivel") {
        std::cout << "Lendo nivel\n";
    } else if (tipo == "pressao") {
        std::cout << "Lendo pressao\n";
    } else if (tipo == "vazao") {
        std::cout << "Lendo vazao\n";
    }
}
```

Responda:

1. por que esse código é limitado do ponto de vista de POO;
2. qual problema de manutenção aparece quando novos sensores são adicionados;
3. como uma classe-base `Sensor` com método virtual poderia melhorar o desenho;
4. que tipo de teste ficaria mais simples depois da refatoração.

Questão 8. Revisão de código em Python: validação fraca do contrato

Analise o código.

```
def processar_leitura(dados):
    tag = dados["tag"]
    valor = dados["valor"]
    print(f"{tag}: {valor}")
```

Responda:

1. quais falhas podem ocorrer se o JSON vier incompleto ou com tipo errado;
2. por que esse código mistura processamento com confiança excessiva na entrada;
3. como separar validação de contrato e uso da leitura;
4. escreva, em alto nível, três casos de teste para essa função ou para uma função de validação associada.

Questão 9. Strategy aplicado à regra de controle

Uma equipe implementou a regra de controle assim:

```
if (modo == "normal") {  
    if (nivel < 30.0) ligarBomba();  
} else if (modo == "economico") {  
    if (nivel < 20.0) ligarBomba();  
} else if (modo == "emergencia") {  
    bloquearPartida();  
}
```

Responda:

1. qual problema de acoplamento aparece nesse desenho;
2. por que `Strategy` pode ser uma solução melhor;
3. como o uso de `Strategy` facilita testes unitários;
4. em que situação essa refatoração seria exagerada.

Questão 10. Command e rastreabilidade de atuação

No projeto, comandos como `LIGAR_BOMBA`, `DESLIGAR_BOMBA` e `BLOQUEAR_PARTIDA` podem ser modelados como objetos.

Explique:

1. qual problema o padrão `Command` resolve;
2. por que ele ajuda a registrar histórico de atuação;
3. como testar um comando pelo efeito causado na bomba;
4. por que um comando deve preservar estado válido, por exemplo impedindo `ligada = true` e `bloqueada = true` ao mesmo tempo.

Questão 11. Repository e persistência testável

Analise o trecho.

```
def mostrar_tela():  
    conexao = sqlite3.connect("leituras.db")  
    cursor = conexao.execute("SELECT * FROM leituras")  
    linhas = cursor.fetchall()  
    st.table(linhas)
```

Responda:

1. por que misturar Streamlit, SQL e regra de apresentação dificulta manutenção;
2. como o padrão `Repository` melhora esse desenho;
3. por que um repositório em memória é útil nos testes;
4. qual teste ainda precisa usar SQLite real, mesmo existindo repositório em memória.

Questão 12. Observer, eventos e alarmes

Explique a diferença entre leitura, evento e alarme no projeto da estação.

Depois, responda:

1. por que `Observer` reduz acoplamento entre quem detecta uma condição e quem reage a ela;
2. dê dois observadores possíveis para um evento de pressão alta;
3. qual risco aparece se a própria classe `SensorPressao` começar a gravar banco, mostrar tela e enviar notificação;
4. que teste poderia provar que pressão alta gera alarme.

Questão 13. Tratamento de erro e contrato entre C++ e Python

Considere que o controlador C++ encontrou uma falha em um sensor. Duas equipes propuseram saídas diferentes:

Equipe A:

```
programa encerra com erro no terminal
```

Equipe B:

```
{"tag": "PT-201", "valor": null, "unidade": "bar", "timestamp": "2026-06-09T10:30:00-03:00", "status": "falha", "erro": "leitura fora da faixa"}
```

Compare as duas soluções e responda:

1. qual delas é mais adequada para integração com o supervisor;
 2. quais cuidados o Python precisa ter ao consumir esse pacote;
 3. por que `valor = null` pode ser aceitável apenas quando o contrato define essa regra;
 4. que teste de contrato deveria existir para esse caso.
-

5. Questões avançadas: arquitetura, testes e entrega profissional

Questão 14. UML do mini-SCADA com padrões de projeto

Modele, em UML, uma versão simplificada do projeto final com as seguintes classes ou equivalentes:

- `Sensor` ;
- `SensorNivel` ;
- `SensorPressao` ;
- `Bomba` ;
- `EstrategiaControle` ;
- `ControleNormal` ;
- `Comando` ;
- `LigarBomba` ;
- `RepositorioLeituras` ;
- `GerenciadorAlarmes` ;
- `Supervisor` .

Seu diagrama deve indicar:

1. pelo menos uma generalização;
2. pelo menos uma realização/interface ou classe abstrata;
3. pelo menos uma associação;
4. multiplicidades quando fizer sentido;
5. quais partes representam `Strategy` , `Command` e `Repository` .

Ao final, justifique duas decisões de modelagem.

Questão 15. Matriz de testes para o projeto final

Monte uma matriz mínima de testes para uma dupla cujo projeto possui:

- regra de nível baixo;
- regra de pressão alta;
- alarme de vazão baixa;
- contrato JSON;
- histórico em SQLite;
- tela Streamlit.

Para cada item, indique:

1. tipo de teste;
2. entrada ou cenário;
3. resultado esperado;
4. se deve rodar localmente, no CI ou manualmente.

Depois, explique por que não basta testar apenas a interface.

Questão 16. Qualidade de código e Definition of Done

Uma dupla abriu um pull request com a seguinte descrição:

Arrumei a parte da bomba.

O PR não informa testes, não atualiza README, não registra uso de IA e altera arquivos do controlador, do supervisor e do banco ao mesmo tempo.

Produza uma revisão técnica explicando:

1. por que esse PR é difícil de revisar;
2. quais evidências mínimas deveriam aparecer;
3. como a dupla poderia dividir melhor o trabalho;
4. como a **Definition of Done** da disciplina ajuda a evitar esse problema.

Questão 17. Decisão arquitetural: arquivo JSON Lines ou TCP

O projeto permite que o controlador C++ entregue leituras ao supervisor por arquivo JSON Lines ou por TCP.

Compare as duas abordagens considerando:

1. simplicidade de implementação;
2. facilidade de teste;
3. proximidade com sistemas reais;
4. tratamento de falhas;
5. adequação para uma entrega de disciplina.

Feche com uma recomendação prática para uma dupla que ainda está com dificuldade em testes e persistência.

Questão 18. Questão-síntese: diagnóstico de um projeto quase pronto

Você recebeu o seguinte relato de uma equipe:

"Nosso mini-SCADA já mostra gráfico no Streamlit. O C++ gera um arquivo com leituras. Ainda não temos testes. O JSON às vezes muda de formato quando há erro. A regra de ligar a bomba está dentro do `main.cpp`. O SQLite é acessado diretamente no arquivo da tela. Os alarmes são uma lista global em Python. O README tem prints, mas não tem comandos para executar."

Escreva uma resposta dissertativa estruturada que:

1. identifique os principais riscos técnicos;
2. conecte cada risco a um conceito da Parte 2;
3. proponha um plano de correção em etapas;
4. indique quais testes devem ser criados primeiro;
5. explique como a equipe deveria preparar a defesa técnica.

6. Questões extras de fixação rápida

Estas questões são curtas e ajudam a revisar vocabulário antes da prova.

1. Qual é a diferença entre `status = "alerta"` e um alarme registrado?
2. Por que `timestamp` em UTC reduz ambiguidade em dados históricos?
3. O que um teste de regressão protege?
4. Por que `Repository` não é apenas "uma classe para SQL"?
5. Qual é o risco de aceitar qualquer JSON porque `json.loads()` não falhou?
6. Em que momento uma exceção deve ser capturada no controlador C++?
7. Por que `Observer` combina bem com alarmes?
8. O que significa dizer que CI não é produção?
9. Por que uma tela bonita não prova boa arquitetura?
10. Qual evidência mostra que uma dupla trabalhou de forma colaborativa?

7. Fontes de referência

- [C++ Core Guidelines](#)

- [cppreference - Classes](#)
- [cppreference - Virtual functions](#)
- [Python Docs - Classes](#)
- [Python Docs - Errors and Exceptions](#)
- [Python Docs - sqlite3](#)
- [Python Docs - json](#)
- [SQLite Docs](#)
- [pytest Docs](#)
- [Streamlit Docs - App testing](#)
- [GitHub Docs - About pull requests](#)
- [GitHub Docs - GitHub Actions](#)